

Лабораторная работа №3: Базовые растровые алгоритмы

В этой работе было задание реализовать приложение, иллюстрирующее работу базовых алгоритмов растеризации отрезков и окружностей. Я реализовала все необходимые алгоритмы, они объединены в приложении, написанном на java.

1. Пошаговый алгоритм
2. Алгоритм ЦДА (Digital Differential Analyzer)
3. Алгоритм Брезенхема для линий
4. Алгоритм Брезенхема для окружностей
5. Алгоритм Кастла-Питвея для растеризации отрезков.
6. Алгоритм Ву для сглаживания линий.

Описание алгоритмов

1. Пошаговый алгоритм

Наиболее очевидный метод, который использует уравнения прямой $y = mx + b$. Алгоритм итерируется по доминирующей оси (оси с большей протяженностью отрезка) и для каждого шага вычисляет координату по другой оси.

Алгоритм (для доминирующей оси X):

$$m = (y_1 - y_0) / (x_1 - x_0)$$
$$b = y_0 - m * x_0$$

для x от x_0 до x_1 :

$$y = \text{round}(m * x + b)$$

установить_пиксель(x , y)

2. Алгоритм ЦДА

Это усовершенствование пошагового алгоритма, заменяющее умножение в цикле на инкрементное сложение. Так как пошаговый алгоритм считается медленным, эта оптимизация достаточно сильно влияет на время выполнения. Однако по-прежнему использует арифметику с плавающей точкой.

Алгоритм:

$$dx = x_1 - x_0$$
$$dy = y_1 - y_0$$
$$steps = \max(|dx|, |dy|)$$

$$x_inc = dx / steps$$
$$y_inc = dy / steps$$

```

x = x0, y = y0
для i от 0 до steps:
    установить_пиксель(round(x), round(y))
    x += x_inc
    y += y_inc

```

3. Алгоритм Брезенхема (линия)

Высокоэффективный алгоритм, использующий только целочисленную арифметику. На каждом шаге принимается решение о смещении по второй оси на основе накопленной “ошибки” — целочисленного параметра, показывающего отклонение растровой линии от идеальной. Самый быстрый алгоритм для растеризации линий благодаря отсутствию операций с плавающей точкой и делений.

Алгоритм (целочисленный, для $|\text{slope}| \leq 1$):

```

dx = |x1 - x0|
dy = |y1 - y0|
error = 2 * dy - dx

```

```

для x от x0 до x1:
    установить_пиксель(x, y)
    если error >= 0:
        y += 1
        error -= 2 * dx
    error += 2 * dy

```

4. Алгоритм Брезенхема (окружность)

Основан на 8-кратной симметрии окружности. Алгоритм вычисляет пиксели только для одного октанта, остальные вычисляются из соображений симметрии.

Алгоритм:

```

x = 0, y = radius
d = 3 - 2 * radius // Начальный параметр решения

```

```

пока x ≤ y:
    установить_8_симметричных_точек(xс, yc, x, y)
    если d < 0:
        d += 4 * x + 6
    иначе:
        d += 4 * (x - y) + 10
        y -= 1
    x += 1

```

5. Алгоритм Ву

Антиалиасинг (сглаживание). Вместо выбора одного пикселя, алгоритм закрашивает два соседних пикселя с разной интенсивностью (прозрачностью), обратно пропорциональной их расстоянию до идеальной линии.

Алгоритм:

```
# Для каждого шага по доминирующей оси
y_ideal = ... // точное значение y
y_int = int(y_ideal)
y_frac = frac(y_ideal) // дробная часть

// Интенсивность обратно пропорциональна расстоянию
установить_пиксель(x, y_int, 1 - y_frac)
установить_пиксель(x, y_int + 1, y_frac)
```

6. Алгоритм Кастла-Питвея

Основан на идее, схожей с алгоритмом Евклида для нахождения НОД. Алгоритм кодирует растровую линию как последовательность прямых (s) и диагональных (d) шагов, которую затем рекурсивно генерирует. Считается менее эффективным, чем Брезенхем, из-за работы со строками. **Алгоритм:**

```
// Преобразовать отрезок (0,0)->(a,b)
y_c = b, x_c = a - b
m1 = "s", m2 = "d"

пока x_c != y_c:
    если x_c > y_c:
        x_c -= y_c
        m2 = m1 + m2
    иначе:
        y_c -= x_c
        m1 = m2 + m1

результатирующая_строка = (m2 + m1) * x_c
```

Вывод по производительности:

Даже для одной и той же линии при перезапуске алгоритма он показывает разные результаты. Это заметно, так как сама по себе работа алгоритмов очень быстрая, и влияние среды очень заметно. Так, в среднем первый запуск медленнее, чем последующие.

Пошаговый алгоритм и Брезенхем (линия) отработали за примерно одинаковое время и показали наилучший результат, близкий к ним результат показывает Кастл-Питвей

ЦДА отработал заметно медленнее. Отдельно стоит отметить, что это замедление при тестировании объясняется не особенностями алгоритма, а выводом большого количества отладочной информации в консоль.

Алгоритм Ву, предназначенный для сглаживания линий, оказался самым тяжёлым по вычислительным затратам, что соответствует его назначению: для каждого шага он обрабатывает два пикселя с различной интенсивностью, что значительно увеличивает количество вычислений.

Реализация

Все алгоритмы и пользовательский интерфейс реализованы в файле `RasterAlgorithmsApp.java`. Вот функции с основной логикой алгоритмов:

- `public List ddaLine(int x1, int y1, int x2, int y2)`
- `public List bresenhamLine(int x0, int y0, int x1, int y1)`
- `public List bresenhamCircle(int xc, int yc, int r)`
- `public List castlePitteway(int x1, int y1, int x2, int y2)`
- `public List wuAntialiasingLine(int x1, int y1, int x2, int y2)`

Пример работы приложения







